

Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (RAG) enhances LLM responses by grounding them in external knowledge. Rather than relying solely on information baked into model weights at training time, a RAG system first retrieves relevant documents from a knowledge base, then conditions generation on that evidence.

The pipeline has three stages. First, a corpus of documents is chunked into smaller fragments and converted into dense vector embeddings using an encoder model. These embeddings go into a vector database that supports approximate nearest-neighbor search. Second, when a user submits a query, the same encoder converts it into a vector. The database returns the top-k most similar chunks by cosine similarity. Third, the retrieved chunks are injected into the LLM prompt as context, and the model generates an answer grounded in that evidence.

RAG addresses two core limitations of parametric-only LLMs: knowledge staleness (the model cannot know events after its training cutoff) and hallucination (it may generate plausible but wrong facts). By attaching retrieved evidence to the prompt, responses can be grounded in up-to-date, verifiable sources.

Key design choices: chunk size (larger chunks preserve context but dilute similarity scores), embedding model (general-purpose vs. domain-specific), k retrieved chunks (too few misses evidence; too many overwhelms the context window), and whether to use sparse retrieval (BM25), dense retrieval (vector search), or hybrid approaches.

RAG is now a standard pattern in enterprise AI, powering knowledge-base Q&A, document search, code assistants, and customer support automation.

Vector Embeddings and Semantic Search

Vector embeddings map discrete objects into a continuous high-dimensional space where semantic similarity corresponds to geometric proximity. For text, a well-trained model places sentences with similar meaning near each other regardless of surface-level word overlap.

The standard model for sentence embeddings in lightweight applications is all-MiniLM-L6-v2, a 384-dimensional model distilled from larger transformers. It performs well on semantic textual similarity benchmarks and is small enough to run on CPU with sub-millisecond inference per sentence. The L6 refers to six transformer layers, a reduction from 12 in BERT that trades a small quality margin for 2x speed.

Cosine similarity is the standard metric for comparing sentence embeddings. It measures the angle between two vectors rather than their Euclidean distance, making it invariant to vector magnitude. A cosine similarity of 1.0 means identical direction; 0.0 means orthogonal; -1.0 means opposite. Cosine distance is 1 minus cosine similarity, so lower is more similar.

In practice, embeddings are L2-normalized before storage so cosine similarity reduces to a dot product. This matters because dot products can be accelerated with BLAS routines and GPU matrix multiplication, enabling vector databases to search billions of embeddings efficiently.

HNSW (Hierarchical Navigable Small World) indexes allow sub-linear search time. HNSW builds a layered graph where each node connects to its nearest neighbors at multiple granularity levels. During search, the algorithm descends through layers following the edge that minimizes distance to the query, achieving query times of microseconds to milliseconds for million-scale corpora.

Multi-Agent Systems and Orchestration

A multi-agent system decomposes a complex task across multiple specialized AI agents, each with a focused role, communicating through well-defined interfaces. This mirrors how organizations divide labor: a coordinator decides who handles what, while specialists execute with domain expertise.

The coordinator agent is responsible for intent classification: determining whether a query is best handled by a knowledge-retrieval agent, a tool-using agent, or a general reasoning agent. A well-designed coordinator produces a structured routing decision that includes explicit reasoning. Not just "route to retriever" but "this question asks about a specific claim in the uploaded documents, which requires semantic search over the knowledge base rather than parametric knowledge." That reasoning is what makes the system auditable.

The retrieval agent specializes in document-grounded question answering. It embeds the user query, searches the vector store for relevant chunks, and constructs an answer that cites the retrieved evidence. Crucially, it surfaces the retrieved context in its output. A system that uses retrieved evidence silently provides no transparency about its information sources.

The general agent handles open-domain reasoning, arithmetic, and current-event queries. It has access to tools: a calculator for mathematical expressions and a web search tool for information outside the knowledge base. Tool calls should use validated Pydantic models, not free-form text, so they are type-safe and auditable.

A common failure mode is fake separation: a single LLM playing multiple roles via system prompt. True separation means each agent has independent configuration, independent tool access, and independent context. Changes to one agent should not affect another.

Federated Learning for Privacy-Preserving AI

Federated learning (FL) is a distributed machine learning paradigm in which a global model is trained across many clients without centralizing raw data. Each client trains locally on its private data and shares only model weight updates with a central server. The server aggregates these updates, typically by weighted averaging in the FedAvg algorithm, and sends the improved global model back to clients.

The main motivation is data privacy. In healthcare, financial services, and mobile computing, raw data is either legally protected (GDPR, HIPAA) or commercially sensitive. Federated learning lets institutions collaborate on model improvement without exposing underlying records. A hospital can contribute to a disease prediction model without sending patient data off-premises.

The Flower (flwr) framework provides a Python implementation of federated learning that abstracts communication protocol, client management, and aggregation strategy. A Flower deployment has a server that coordinates training rounds and clients that each wrap a local training loop. Clients implement two methods: `fit` (local training, returns updated weights) and `evaluate` (local validation, returns loss and metrics).

Key challenges include statistical heterogeneity (client data distributions differ, causing drift toward clients with more data), communication efficiency (transmitting large model updates is bandwidth-intensive), and privacy leakage (gradient updates can expose training data through model inversion attacks; differential privacy and secure aggregation address this).

Federated learning combined with RAG is an emerging research direction: clients collectively improve a shared retrieval model without centralizing the documents they search, enabling privacy-preserving knowledge base construction for sensitive enterprise domains.